

Reviewer Pack — Minimal Geometric Entropy of Tropical Curves in Tseitin Form...

Entry 3aff2c93a381 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 00:28:23 UTC

Minimal Geometric Entropy of Tropical Curves in Tseitin Formula Resolution Proofs

Entry ID: 3aff2c93a381 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 00:28:23 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution Proof Complexity for Tseitin Formulas

Statement:

[‘For a given Tseitin formula F on an expander graph G , the minimal geometric entropy of its defining tropical curve $H(G)$ is lower-bounded by the Resolution refutation length of F as $H(G) = \Theta(2^{L(F)})$.’, ‘For all Tseitin formulas F on non-expanders, the geometric entropy $H(G)$ satisfies $H(G) = O(1)$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a rich framework for studying algebraic structures in graphs, and its geometric entropy is expected to capture essential features of the graph that affect complexity.’, ‘Expander graphs are known to be hard for Resolution proofs, suggesting a potential connection between their tropical geometric properties and their proof complexity.’]

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 86cb49d8c15f80f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across at least 24 of 30 seeds, the ratio of the minimal geometric entropy $H(G)$ to the Resolution refutation length $L(F)$ is greater than or equal to 0.5 AND for all Tseitin formulas on non-expanders, the geometric entropy $H(G)$ does not exceed 1.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND resolution proof complexity Tseitin formula - geometric entropy tropical curves Resolution refutation length - minimal geometric entropy Tseitin formulas expander graph

Top relevant hits considered: - [<http://arxiv.org/abs/1206.1925v1>] Counting Algebraic Curves with Tropical Geometry - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/2309.17302v2>] Geometry of tropical extensions of hyperfields - [<http://arxiv.org/abs/1812.08038v3>] On refined count of rational tropical curves - [<http://arxiv.org/abs/math/0511713v2>] Tropical Plane Geometric Constructions: a Transfer Technique in Tropical Geometry - [<http://arxiv.org/abs/1505.04658v2>] A survey of recent results in (generalized) graph entropies - [<http://arxiv.org/abs/2002.11069v2>] Minimal volume entropy of simplicial complexes - [<http://arxiv.org/abs/1908.05124v2>] Minimal Representations of Order Types by Geometric Graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A
```

```

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B[0]), len(B)
    C = [[0 for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m = len(A)
    if m == 1:
        return A[0][0]
    det = 0
    for i in range(m):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += (-1) ** i * A[0][i] * determinant(submatrix)
    return det

def is_expander(G, n):
    degree_sum = sum(sum(1 for _ in neighbors) for neighbors in G)
    if degree_sum == 0:
        return False
    avg_degree = degree_sum / n
    if avg_degree < 2:
        return False
    max_degree = max(len(neighbors) for neighbors in G)
    if max_degree > 2 * avg_degree:
        return True
    return False

def geometric_entropy(G):
    n = len(G)
    A = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i+1, n):
            if random.choice([True, False]):
                A[i][j] = 1
                A[j][i] = 1
    det_A = determinant(A)
    if det_A == 0:
        return float('inf')
    entropy = -math.log2(det_A / n**n)
    return entropy

def tseitin_formula(G):
    n = len(G)
    formula = []
    for i in range(n):
        clause = [f'x{i}', f'y{i}']
        formula.append(clause)
        for j in range(i+1, n):
            if G[i][j]:
                clause = [f'x{j}', f'y{i}', f'y{j}']
                formula.append(clause)
    return formula

```

```

def resolution_refutation_length(formula):
    length = 0
    while True:
        new_clauses = []
        for i in range(len(formula)):
            for j in range(i+1, len(formula)):
                if set(formula[i]).isdisjoint(set(formula[j])):
                    continue
                common_vars = list(set(formula[i]) & set(formula[j]))
                for var in common_vars:
                    new_clause = [f'~{var}']
                    for v in formula[i]:
                        if v != var and v[0] == '~':
                            new_clause.append(v[1:])
                    for v in formula[j]:
                        if v != var and v[0] != '~':
                            new_clause.append('~' + v)
                    new_clauses.append(new_clause)
        if not new_clauses:
            break
        formula.extend(new_clauses)
        length += 1
    return length

n = random.randint(5, 40)
G = [[random.choice([True, False]) for _ in range(n)] for _ in range(n)]
G = [row[:] for row in G]
for i in range(n):
    G[i][i] = True
if not is_expander(G, n):
    return {
        "metric_name": "geometric_entropy",
        "metric_value": 0,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "non-expander_graph"
    }

H = geometric_entropy(G)
F = tseitin_formula(G)
L_F = resolution_refutation_length(F)

return {
    "metric_name": "geometric_entropy",
    "metric_value": H / L_F,
    "instances_tested": 1,
    "conjecture_holds": H >= 0.5 * L_F,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(res["metric_value"] for res in results) / len(results)
std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"geometric_entropy < 0.5 * resolution_refutation\")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

-expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}
TRIAL: {'metric_name': 'geometric_entropy', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': True, 'expander_graph': 'expander_graph'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7780dc.py", line 161, in <module>
    first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)
    ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1e7780dc.py", line 161, in <genexpr>
    first_failing_seed = next((res["seed"] for res in results if not res["conjecture_holds"]), None)
    ~~~~~^~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Re-run the test without crashing to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14108
2	preregistration	ollama_remote	glm4:latest	0	0	9582
3	novelty	ollama_remote	glm4:latest	0	0	8174
4	novelty	ollama_remote	glm4:latest	0	0	9480
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12552
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10314
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19693
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15542
9	judge	ollama_remote	glm4:latest	0	0	12470

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111914 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3aff2c93a381.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3aff2c93a381.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3aff2c93a381.tar.gz` (if generated)